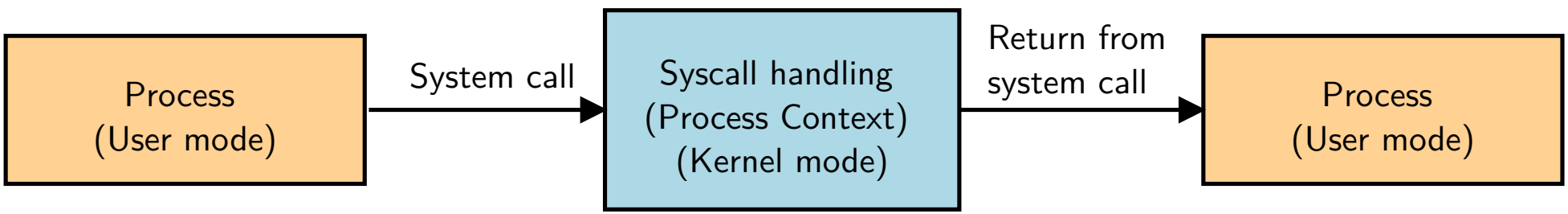


The process context



Process context

- The *process context* can be seen as the content of the CPU registers associated to a process: execution register, stack register...
- This context also designates an execution state and allows to sleep inside kernel mode.
- A process that is executing in process context can be preempted.
- While executing in such context, the current process struct task_struct can be accessed using get_current().





Scheduling



Scheduling

- The scheduler can be invoked for various reasons
  - On a periodic tick caused by interrupt (CONFIG_HZ)
  - On a programmed interrupt on tickless systems (CONFIG_NO_HZ=y)
  - Voluntarily by calling schedule() in code
  - Implicitly by calling functions that can sleep (blocking operations such as kmalloc(), wait_event()).
- When entering the schedule function, the scheduler will elect a new struct task_struct to run and will eventually call the switch_to() macro.
- switch_to() is defined by architecture code and it will save the current task process context and restore the one of the next task to be run while setting the new current task running.



The Linux Kernel Scheduler

- The Linux Kernel Scheduler is a key piece in having a real-time behaviour
- It is in charge of deciding which **runnable** task gets executed
- It also elects on which CPU the task runs, and is tightly coupled to CPUidle and CPUFreq
- It schedules both **userspace** tasks and **kernel** tasks
- Each task is assigned one **scheduling class** or **policy**
- The class determines the algorithm used to elect each task
- Tasks with different scheduling classes can coexist on the system



Non-Realtime Scheduling Classes

There are 3 **Non-RealTime** classes

- `SCHED_OTHER`: The default policy, using a time-sharing algorithm
  - This policy is actually called SCHED_NORMAL by the kernel
- SCHED_BATCH: Similar to `SCHED_OTHER`, but designed for CPU-intensive loads that affect the wakeup time
- SCHED_IDLE: Very low priority class. Tasks with this policy will run only if nothing else needs to run.
- `SCHED_OTHER` and SCHED_BATCH use the **nice** value to increase or decrease their scheduling frequency
  - A higher nice value means that the tasks gets scheduled **less** often



Realtime Scheduling Classes

There are 3 **Realtime** classes

- Runnable tasks will preempt any other lower-priority task
- SCHED_FIFO: All tasks with the same priority are scheduled **First in, First out**
- SCHED_RR: Similar to SCHED_FIFO but with a time-sharing round-robin between tasks with the same priority
- Both SCHED_FIFO and SCHED_RR can be assigned a priority between 1 and 99
- SCHED_DEADLINE: For tasks doing recurrent jobs, extra attributes are attached to a task
  - A computation time, which represents the time the task needs to complete a job
  - A deadline, which is the maximum allowable time to compute the job
  - A period, during which only one job can occur
- Using one of these classes is necessary but not sufficient to get real-time behavior



Changing the Scheduling Class

- The Scheduling Class is set per-task, and defaults to `SCHED_OTHER`
- The man 2 sched_setscheduler syscall allows changing the class of a task
- The `chrt` tool uses it to allow changing the class of a running task:
  - `chrt -f/-b/-o/-r/-d -p PRIO PID`
- It can also be used to launch a new program with a dedicated class:
  - `chrt -f/-b/-o/-r/-d PRIO CMD`
- To show the current class and priority:
  - `chrt -p PID`
- New processes will inherit the class of their parent except if the SCHED_RESET_ON_FORK flag is set with man 2 sched_setscheduler
- See man 7 sched for more information



Execution mode switching



Execution mode switching

- Execution mode switching is the action of changing the execution mode of the processor (Kernel ↔ User).
  - Explicitly by executing system calls instructions (synchronous request to the kernel from user mode).
  - Implicitly when receiving exceptions (MMU fault, interrupts, breakpoints, etc).
- This state change will end up in a kernel entrypoint (often call vectors) that will execute necessary code to setup a correct state for kernel mode execution.
- The kernel takes care of saving registers, switching to the kernel stack and potentially other things depending on the architecture.
  - Does not use the user stack but a specific kernel fixed size stack for security purposes.



Exceptions

- Exceptions designate the kind of events that will trigger a CPU execution mode change to handle the exception.
- Two main types of exceptions exist: synchronous and asynchronous.
  - Asynchronous exceptions when a fault happens while executing (MMU, bus abort, etc) or when an interrupt is received (either software or hardware).
  - Synchronous when executing some specific instructions (breakpoint, syscall, etc)
- When such exception is triggered, the processor will jump to the exception vector and execute the code that was setup for this exception.



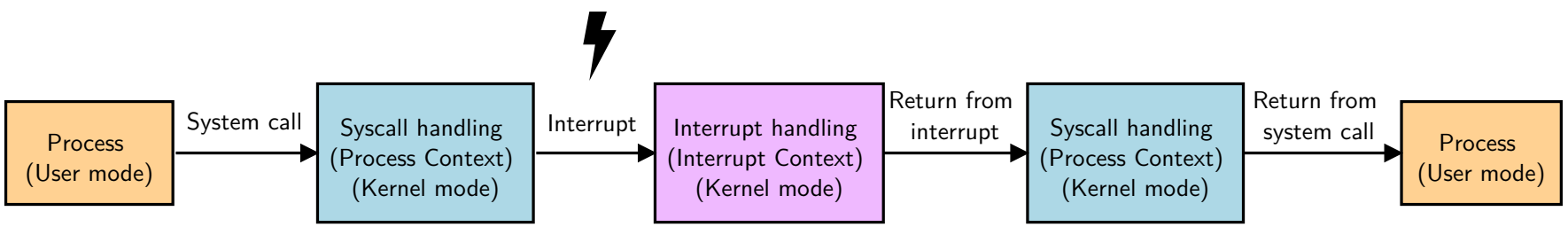
Interrupts

- Interrupts are asynchronous signals that are generated by the hardware peripherals.
  - Can also be synchronous when generated using a specific instruction (**I**nter **P**rocessor **I**nterrupts for instance).
- When receiving an interrupt, the CPU will change its execution mode by jumping to a specific vector and switching to kernel mode to handle the interrupt.
- When multiple CPUs (cores) are present, interrupts are often directed to a single core.
- This is called "IRQ affinity" and it allows to control the IRQ load for each CPU
  - See core-api/irq/irq-affinity and man irqbalance(1)



Interrupt context

- While handling the interrupts, the kernel is executing in a specific context named *interrupt context*.
- This context does not have access to userspace and should not use get_current().
- Depending on the architecture, might use an IRQ stack.
- Interrupts are disabled (no nested interrupt support)!





System Calls (1/2)

- A system call allows the user space to request services from the kernel by executing a special instruction that will switch to the kernel mode (man 2 syscall)
  - When executing functions provided by the libc (`read()` , `write()` , etc), they often end up executing a system call.
- System calls are identified by a numeric identifier that is passed via the registers.
  - The kernel exports some defines (in `unistd.h`) that are named `__NR_<syscall>` and defines the syscall identifiers.

```
#define __NR_read 63
#define __NR_write 64
```



System Calls (2/2)

- The kernel holds a table of function pointers which matches these identifiers and will invoke the correct handler after checking the validity of the syscall.
- System call parameters are passed via registers (up to 6).
- When executing this instruction the CPU will change its execution state and switch to the kernel mode.
- Each architecture uses a specific hardware mechanism (man 2 syscall)

```
mov w8, #__NR_getpid
svc #0
tstne x0, x1
```



Kernel execution contexts



Kernel execution contexts

- The kernel runs code in various contexts depending on the event it is handling.
- Might have interrupts disabled, specific stack, etc.



Kernel threads

- Kernel threads (kthreads) are a special kind of struct task_struct that do not have any user resources associated (`mm == NULL`).
- These processes are cloned from the `kthreadd` process and can be created using kthread_create().
- Kernel threads are scheduled and are allowed to sleep much like a process executing in process context.
- Kernel threads are visible and their names are displayed between brackets under *ps*:

```
$ ps --ppid 2 -p 2 -o uname,pid,ppid,cmd,cls
USER         PID    PPID CMD                         CLS
root           2       0 [kthreadd]                    TS
root           3       2 [rcu_gp]                      TS
root           4       2 [rcu_par_gp]                  TS
root           5       2 [netns]                       TS
root           7       2 [kworker/0:0H-events_highpr   TS
root          10       2 [mm_percpu_wq]                TS
root          11       2 [rcu_tasks_kthread]           TS
```



Workqueues

- Workqueues allows to schedule some work to be executed at some point in the future
- Workqueues are executing the work functions in kernel threads.
  - Allows to sleep while executing the deferred work.
  - Interrupts are enabled while executing
- Work can be executed either in dedicated work queues or in the default workqueue that is shared by multiple users.

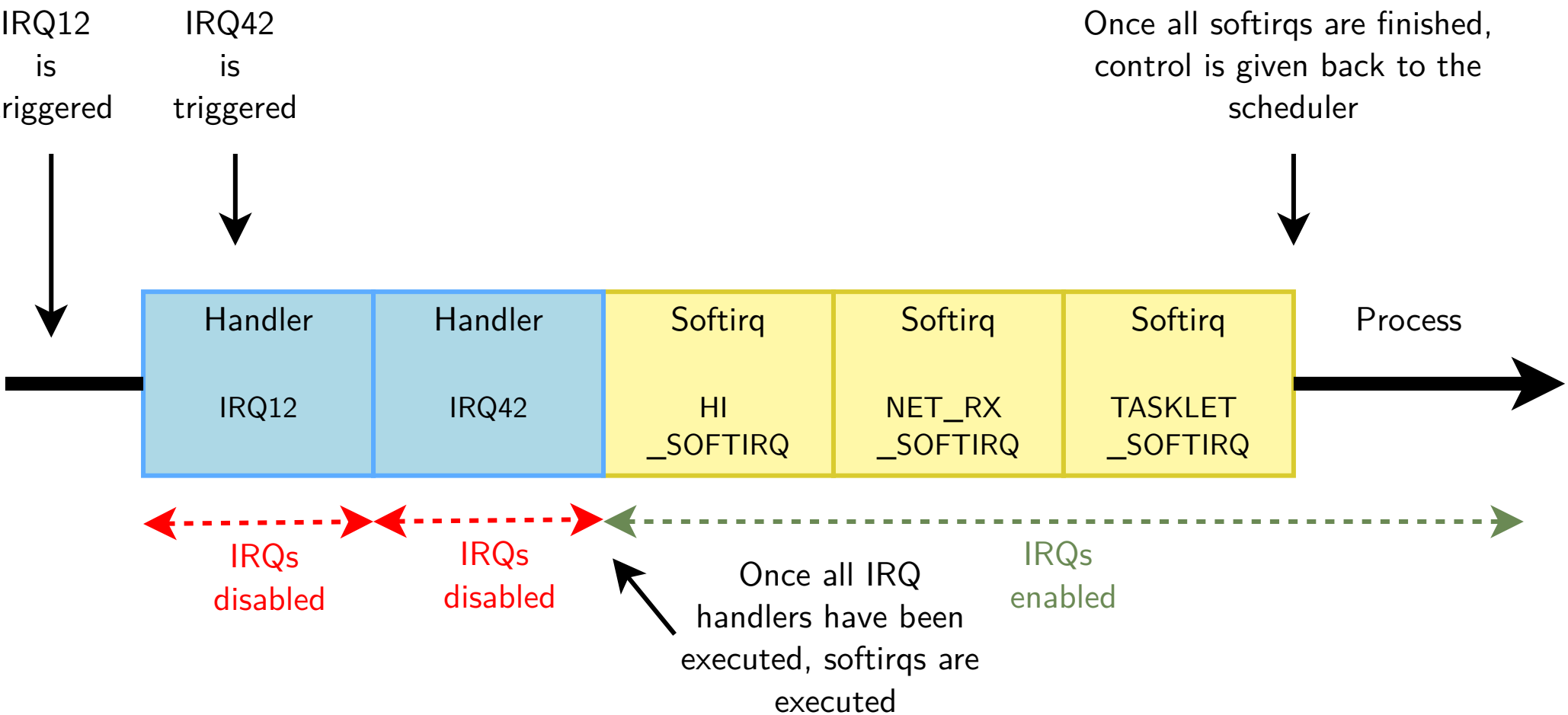


softirq

- SoftIRQs is a specific kernel mechanism that is executed in software interrupt context.
- Allows to execute code that needs to be deferred after interrupt handling but needs low latency.
  - Executed right after hardware IRQ have been handled in interrupt context.
  - Same context as executing interrupt handler so sleeping is not allowed.
- Anyone wanting to run some code in softirq context should likely not create its own but prefer some entities implemented on top of it. There are for example tasklets, and the BH workqueues (Bottom Half workqueues) which aim to replace tasklets since 6.9.



Interrupts & Softirqs





Threaded interrupts

- Threaded interrupts are a mechanism that allows to handle the interrupt using a hard IRQ handler and a threaded IRQ handler.
  - Created calling request_threaded_irq() instead of request_irq()
- A threaded IRQ handler will allow to execute work that can potentially sleep in a kthread.
- One kthread is created for each interrupt line that was requested as a threaded IRQ.
  - *kthread* is named `irq/<irq>-<name>` and can be seen using *ps*.



Allocations and context

- Allocating memory in the kernel can be done using multiple functions:
  - `void *kmalloc(size_t size, gfp_t gfp_mask);`
  - `void *kzalloc(size_t size, gfp_t gfp_mask);`
  - `unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order)`
- All allocation functions take a `gfp_mask` parameter which allows to designate the kind of memory that is needed.
  - GFP_KERNEL: Normal allocation, can sleep while allocating memory (can not be used in interrupt context).
  - GFP_ATOMIC: Atomic allocation, won't sleep while allocating data.

Practical lab - Preparing the system



Prepare the STM32MP157D board

- Build an image using Buildroot
- Connect the board
- Load the kernel from SD card
- Mount the root filesystem over NFS